



개인 프로젝트 · 백엔드 설계 및 구현

정산어택

1차, 2차로 끝나지 않는 술자리를 위한 정산 서비스.

N으로 나누지 않고 차수별 참석과 음주 여부까지 나눠 계산하며, 입력을 총무 혼자 하지 않는다.

기간	역할	형태	상태
2026.08 ~ 진행 중	기획 · 설계 · 개발 단독	웹 + 모바일 앱	계산 엔진 검증 완료

1 왜 만들었나

술자리 정산은 나눗셈 문제가 아니라 기록 문제다.

1차는 다섯 명이 갔는데 2차는 셋만 갔다. 한 명은 술을 안 마셨고, 택시비는 두 명만 나눠 낸다. 여기서 총 금액을 인원수로 나누면 누군가는 반드시 손해를 본다.

그런데 이걸 제대로 계산하려면 누가 어디에 있었는지를 전부 알아야 하고, 그 정보는 총무의 기억 속에만 있다. 기존 정산 앱들이 N빵에 머무는 이유가 여기 있다 — 계산이 어려워서가 아니라 입력을 모을 방법이 없어서다.

문제의 구조

기존 정산 앱

$220,000 \div 5 = 44,000$ 원

2차에 안 간 사람도 44,000원

술을 안 마신 사람도 44,000원

→ 정확하지 않다는 걸 모두가 안다

정산어택

차수별 · 음주 여부별로 분리

총무는 금액만 넣는다

참석·음주는 각자 링크에서 체크

→ 28,300원 ~ 50,470원 으로 갈린다

설계의 중심은 "입력을 분산시키는 것"이다

참여자도 앱을 설치하지 않는다. 단독방 링크로 들어와 웹에서 자기 것만 체크한다. 설치를 요구하는 순간 절반이 이탈하고, 그러면 총무가 다시 전부 입력하게 된다.

2 이 서비스의 난이도는 화면이 아니라 계산에 있다

그리고 그 사실을 측정으로 확인했다.

2.1 BigDecimal 은 이 문제에 안전하지 않다

금액 분배는 나누어떨어지지 않는다. 44,000원을 3명이 나누면 14,666.666...원이다. 통념대로 BigDecimal 로 누적하면 될 것 같지만, 랜덤 시나리오 30,000건을 돌려 참조 구현과 대조한 결과 210건에서 금액이 어긋났다.

30,000건

랜덤 시나리오

인원·차수·금액 무작위

210건

금액 불일치

0.70%

100자리

정밀도를 올려도

해결되지 않음

0건

유리수 구현 후

불일치 없음

정밀도를 100자리로 올려도 사라지지 않는다. 유한 자릿수로 자르는 순간 오차가 생기고, 그 오차가 인원수만큼 누적된 뒤 반올림 경계를 넘으면 금액이 바뀐다. 정밀도의 문제가 아니라 표현의 문제였다.

```
// 원부담을 유리수로 누적한다 - 자르지 않는다
data class Rational(val numerator: BigInteger, val denominator: BigInteger)

// 정수로 바꾸는 곳은 이 함수 하나뿐이다
fun ceilTo(unit: Int): Long =
    (ceilDiv(numerator, denominator * u) * u).longValueExact()
```

2.2 합계가 원금과 어긋날 수 없게 만든다

각자의 몫을 올림하면 합계가 원금보다 커진다. 흔한 해법은 **차액을 계산해 누군가에게 보정해 더하는 것**인데, 이 방식은 보정 로직에 버그가 생기면 합계가 조용히 틀린다.

```
// ★ 나눗셈이 아니라 뺄셈이다.  
// 이 한 줄 때문에 합계가 원금과 어긋날 수가 없고, 보정 로직이 한 줄도 필요 없다.  
amounts[mainPayerId] = grandTotal - amounts.values.sum()
```

대표결제자(가장 많이 결제한 사람)의 몫을 계산하지 않고 뺄셈으로 남긴다. 나머지가 어떻게 반올림되든 **합계는 정**
의상 원금과 같다. 보정 코드가 존재하지 않으므로 보정 버그도 존재할 수 없다.

739줄

계산 엔진 본문

Kotlin · 순수 함수

741줄

테스트

본문과 1:1

46건

테스트 전부 통과

실패 0 · 건너뛴 0

18종

검증 오류 코드

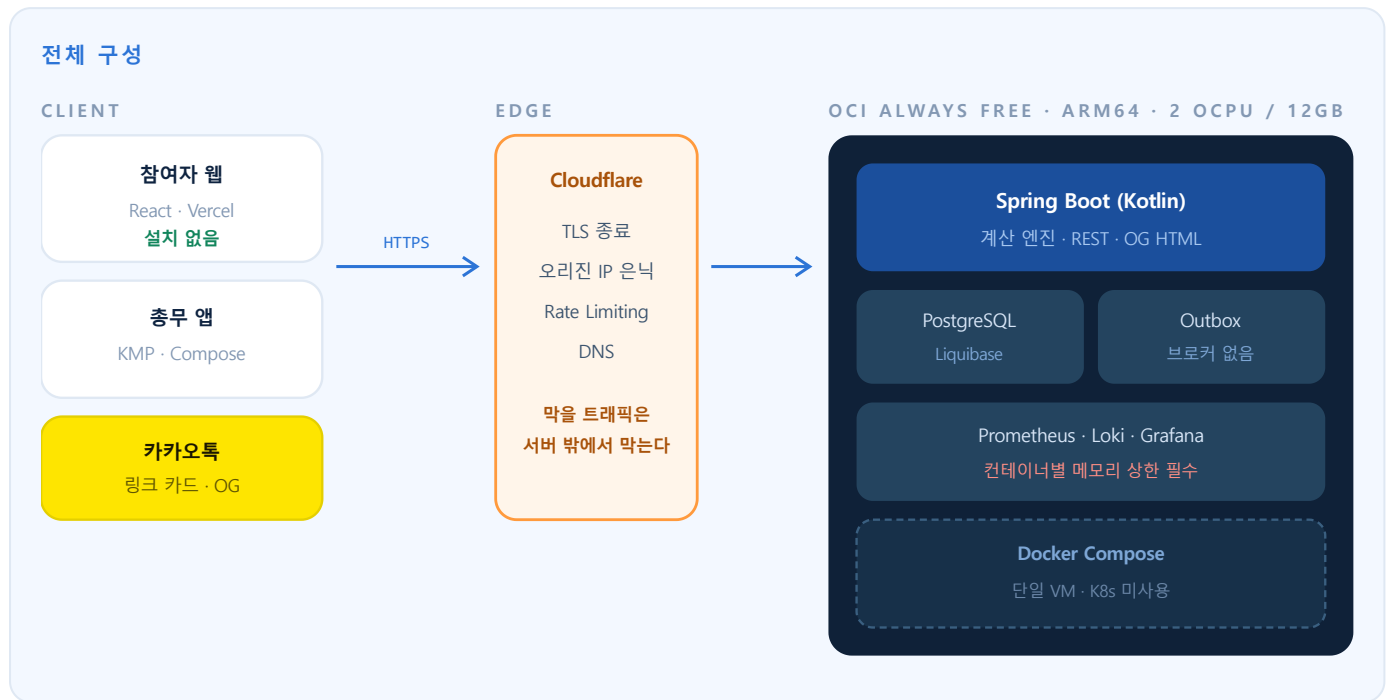
어느 차수인지까지 담는다

계산 결과를 저장하지 않는다

체크 기록으로부터 **조회할 때마다 다시 계산한다**. 저장하면 "저장된 금액"과 "지금 계산한 금액"이 갈라지는 순간이 반드시 오고, 그때 어느 쪽이 맞는지 판단할 근거가 없다. 대신 입금 표시 시점의 부담액만 기록해 **추가금과 환불을 같은 식으로 낸다**.

3 시스템 구성

참여자는 웹, 총무는 앱. 계산은 서버 한 곳에서만 한다.



감시자가 감시 대상을 죽이지 않게 한다

앱·PostgreSQL·Loki·Prometheus·Grafana 를 **12GB 한 대에** 올린다. 상한이 없으면 Prometheus 가 부풀어 앱을 OOM 으로 죽인다. **컨테이너별 메모리 상한과 스왑을** 처음부터 지정하고 앱과 DB 에 우선순위를 준다.

4 기술 스택

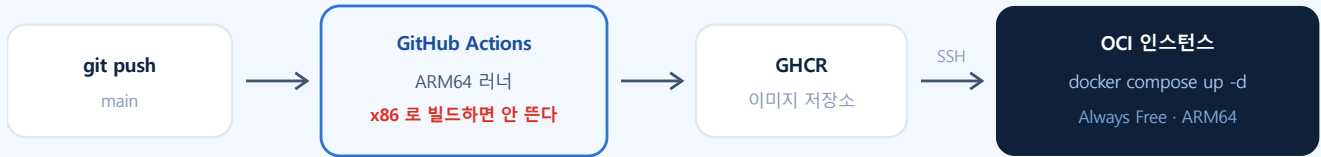
고른 이유가 없는 항목은 넣지 않았다.

영역	선택	고른 이유
언어 · 프레임 워크	Kotlin · Spring Boot	계산 엔진을 순수 함수로 격리하기 좋고, 서버와 앱이 코드를 공유한다
데이터	PostgreSQL · JPA QueryDSL · JdbcTemplate	조회 한 번에 6개 테이블을 조립해야 해 JPA 컬렉션 조인이 불리하다. 그 지점만 JdbcTemplate
스키마	Liquibase (YAML)	ddl-auto: none 고정. 모든 컬럼에 주석을 강제하고 명세를 자동 추출한다
비동기	Outbox 테이블 + 스케줄러	알림 하나 때문에 브로커를 세우면 운영 대상이 하나 더 생긴다 . 트랜잭션과 같이 커밋되는 이점도 크다
테스트	JUnit5 · Kotest · Testcontainers	H2 는 PostgreSQL 과 다르게 동작한다. 잠금·유니크 검증은 실제 엔진에서만 의미가 있다
관측	Actuator · Micrometer Prometheus · Loki · Grafana	단일 인스턴스 자체 호스팅. 지표·로그·추적을 한 화면에서 본다
보안	Bucket4j · Cloudflare	공유 토큰 무작위 대입 방어를 두 겹으로 나눈다 (\$5)
빌드 · 배포	Docker(ARM64) · GitHub Actions GHCR · Docker Compose	OCI Always Free 가 ARM 이라 x86 러너로 빌드하면 뜨지 않는다
프론트	React · TypeScript · Vercel	참여자에게 설치를 요구하지 않기 위한 선택
모바일	Kotlin Multiplatform · Compose	Android · iOS 를 한 코드로. 서버와 언어도 같다

5 배포

무료 한도 안에서 운영하되, 무료라서 생기는 제약을 설계에 반영했다.

배포 파이프라인



5.1 무료 인스턴스가 ARM 이라는 것이 빌드를 바꾼다

OCI Always Free 의 2 OCPU / 12GB 는 **Ampere A1, 즉 ARM64** 다. GitHub Actions 기본 러너는 x86 이므로 그냥 빌드하면 **서버에서 뜨지 않는 이미지**가 나온다. ARM 러너를 쓰거나 **buildx** 크로스 빌드가 필요하다.

5.2 요청 제한을 두 겹으로 나눈다

공유 링크 /g/{token} 은 로그인 없이 응답하는 유일한 주소다. 토큰 문자열 자체가 열쇠이므로 무작위 대입 대상이 된다.

계층	규칙	왜 여기인가
Cloudflare	IP 당 분당 30회	우리 서버에 당기 전에 끊긴다. 12GB 한 대에 모든 것이 올라가므로 이득이 크다
애플리케이션 Bucket4j	연속 404 5회 → 10분 차단	토큰 유효성은 DB 를 봐야 한다. Cloudflare 가 대신할 수 없다

임계값은 정상 사용을 세서 정했다. 링크 하나를 여는 데 2~3회 부른다 — 참여 이후에는 이 주소를 아예 쓰지 않기 때문이다. **성공 응답은 세지 않는다**. 단독방에 뿌리면 같은 사무실 20명이 NAT 뒤 같은 IP 로 들어오므로, 세야 하는 것은 요청 수가 아니라 **틀린 요청 수**다.

토큰 길이가 근본 방어다

12자 · 62종으로 정했다. 7자·36종이면 모임 1만 개일 때 초당 1,000회로 **1.5시간**에 절반이 뚫린다. 12자·62종이면 같은 조건에서 **5,100만 년**이다. 제한은 보조일 뿐이고, 짧은 토큰은 제한으로 메울 수 없다 — 분산하면 IP 제한은 피해간다.

6 클라이언트 배포 — iOS 는 맥 없이

개발 환경이 Windows 라 iOS 배포 경로를 먼저 확인했다.

Kotlin Multiplatform 은 iOS 네이티브 바이너리로 컴파일된다. 어떤 방식으로든 macOS 환경이 필요하다. 맥을 사지 않고 **GitHub Actions** 의 macOS 러너로 빌드해 App Store Connect 에 올린다.

```
# .github/workflows/ios-release.yml
jobs:
  build:
    runs-on: macos-latest # 클라우드 맥. 로컬에 맥이 없어도 된다
    steps:
      - ./gradlew linkReleaseFrameworkIosArm64
      - xcodebuild archive -exportArchive
      - xcrun altool --upload-app # App Store Connect 로 전송
```

	Google Play	App Store
테스터 요건	20명 × 14일 비공개 테스트 신규 개인 개발자 계정 기준	없음 바로 정식 심사로 갈 수 있다
베타 도구	비공개/공개 테스트 트랙	TestFlight — 내부 100명 / 외부 10,000명 첫 외부 빌드만 간단한 베타 심사
빌드 환경	Windows 에서 그대로	macOS 필요 → GitHub Actions macos-latest
계정 비용	25 USD 1회	99 USD / 년

비공개 저장소는 macOS 분이 10배로 차감된다

GitHub Actions 무료 2,000분 기준, macOS 러너는 분당 10배로 계산되어 실질 200분이다. KMP iOS 빌드가 10~20분이면 월 10~20회가 한도다. iOS 빌드는 태그 푸시에서만 돌리고 일반 커밋에서는 돌리지 않는다.

순서는 Android 를 먼저 낸다. 20명 × 14일 요건이 리드타임이 가장 길어 일정을 결정하는 것은 심사가 아니라 이 대기 시간이기 때문이다.

7 진행 상태

검증된 것과 아직인 것을 구분해 적는다.

상태	항목	확인된 근거
완료	계산 엔진	Kotlin 순수 함수 739줄 · 테스트 741줄 · 46건 전부 통과 . 유리수 구현으로 BigDecimal 오차 0건
완료	설계 문서	제품 명세 · 계산 규칙 · API 계약 · 아키텍처 결정 기록 8건 · 총 2,740줄
완료	API 계약	엔드포인트 24개 · 요청/응답 · 오류 코드 34종. 계산 엔진 ErrorCode 와 기계로 대조해 누락 0건 확인
완료	웹 프론트	화면 12개 · 2,483줄 · 실제 도메인에 배포 . 백엔드 연동 전이라 목 데이터로 동작
진행	스키마	Liquibase YAML · ERD · 테이블 명세. 모든 컬럼에 주석을 강제하고 명세를 자동 추출
진행	서버 모듈	계약이 확정되어 그대로 구현하는 단계
계획	모바일 앱	KMP · Android 우선 → iOS

순서를 계산 → 계약 → 구현 으로 잡았다

이 서비스의 난이도는 화면이 아니라 **금액이 맞느냐**에 있다. 그래서 **계산 엔진을 DB-HTTP 없이 먼저 만들어 검증**하고, 프론트와 주고받을 형식을 문서로 못 박은 뒤에 서버를 짠다. 필드 이름 하나가 어긋나도 양쪽을 다시 쓰기 때문이다.

8 검토 중인 확장

제품에 실제로 필요한 것만 넣는다.

8.1 영수증 OCR — 총무의 마지막 수작업을 없앤다

지금 총무는 **총액과 술값을 직접 입력한다**. 그런데 술값을 따로 적는 것이 이 서비스의 핵심인데도 **영수증을 보고 주류 품목을 사람이 골라 더해야 한다**. **OCR 로 품목을 읽으면 그 작업이 사라진다**.

영수증 촬영 → OCR 품목 인식 → 주류 자동 분류 → **총액 · 술값 자동 입력**
총무는 확인만 한다

후보	영수증 특화	판단
NAVER CLOVA OCR	있음	한국 영수증 인식률이 가장 높다. 1순위
Google Cloud Vision	범용 텍스트	품목 구조는 직접 파싱해야 한다. 무료 한도가 넉넉해 대안
AWS Textract AnalyzeExpense	있음	한국어 미지원으로 알려져 있어 후보에서 제외 — 도입 전 재확인 필요

8.2 이미지가 생기면 오브젝트 스토리지가 필요해진다

영수증 사진을 받는 순간 **사용자 업로드 파일**이 생긴다. **12GB 인스턴스 디스크에 이것을 쌓으면 안 된다.** 용량이 예 측되지 않고, 인스턴스를 다시 만들면 사라지며, 백업 대상이 하나 더 늘기 때문이다.

후보	판단
AWS S3	수명주기 정책으로 일정 기간 후 자동 삭제가 쉽다. 영수증은 정산이 끝나면 보관할 이유가 없다
OCI Object Storage	같은 계정 안이라 네트워크 비용이 없다. 인스턴스와 같은 곳에 두는 이점

이유 없이 서비스를 늘리지 않는다
스토리지가 필요한 이유는 영수증 이미지가 생기기 때문이지, 스택을 늘리기 위해서가 아니다. **OCR** 을 넣지 않 기로 하면 이 항목도 같이 빠진다.

9 이 프로젝트에서 다룬 것

주제	내용
정확성을 측정으로 증 명	통념(BigDecimal)을 30,000건 대조로 반증하고 유리수로 교체. 0.70% 불일치 → 0건
불변식을 코드 구조로 보장	대표결제자 뒀을 뿔셈 으로 남겨 합계가 원금과 어긋날 수 없게 만들었다. 보정 로직이 존재하지 않는 다
동시성 설계	경합 4종을 성질별로 나눠 행 잠금 · 복합 유니크 · 조건부 UPDATE 로 각각 처리
계약 우선 개발	엔드포인트 24개와 오류 코드 34종을 구현 전에 확정하고 엔진 enum 과 기계 대조
계약 아래의 인프라 설계	무료 ARM 인스턴스 한 대 · 크로스 빌드 · 컨테이너 메모리 상한 · 옻지와 애플의 방어 분담
결정 기록	ADR 8건. 각 문서에 버린 안과 재검토 조건 을 함께 남겼다

정산어택 프로젝트 소개서 · 수치는 저장소에서 직접 측정한 값이다 · core 739/741줄 · 테스트 46건 · ADR 8건 · 문서 2,740줄 · 화면 12개